

Faculty of Computing

Year 2 Semester 1 (2026)

IT2120 - Probability and Statistics

Lab Sheet 01

Lab Exercise 1 (Introduction to R)

The S language has been developed since the late 1970s by John Chambers and colleagues at Bell Labs as a language for programming with data. This provided an explicit and consistent structure for manipulating, analyzing statistically, and visualizing data.

R is an independent open source implementation similar to S, originally developed by Ross Ihaka and Robert Gentleman at the University of Auckland in the mid-1990s.

R is distributed under the GNU open software license and developed by the user community. It can be downloaded from . <https://cloud.r-project.org/>

R is an integrated suite of software facilities for data manipulation, calculation and graphical display. Among other things it has,

- An effective data handling and storage facility,
- A suite of operators for calculations on arrays, in particular matrices,
- A large, coherent, integrated collection of intermediate tools for data analysis,
- Graphical facilities for data analysis and display either directly at the computer or on hardcopy.

R is a command line driven program, and it is in itself a programming language. *“Many users think of R as a statistics system. We prefer to think of it of an environment within which statistical techniques are implemented.”* says the developers.

Note: When installing R, might come across problems if there are spaces in the path name.

- Latest Version: 4.4.2 - Released in 2024.
- Available On: Windows, Linux, Mac (OS X).
- Terminal and GUI available.
- IDEs for R: R Studio, Rattle.

Basics

- The user can enter commands at the prompt (`>` by default) and each command is executed, one line at a time. Also, commands can be typed in script file and those commands can run one at a time or as a whole.
- Comments can be started with the hash sign (`#`).
- Documentation for objects & functions can be viewed by using the `help(object/function)` or by using `?object/function`.

The Workspace

The workspace is your current R working environment and includes any user-defined objects (vectors, matrices, data frames, lists, functions). At the end of an R session, the user can save an image of the current workspace that is automatically reloaded the next time R is started. Up and down arrow keys scroll through your command history.

Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
^ or **	Exponentiation
%%	Modulus (mod)
%/%	Integer Division

R's binary and logical operators will look very familiar to programmers. Note that binary operators work on vectors and matrices as well as scalars.

Operator	Description
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	exactly equal to
!=	not equal to
!	NOT
	OR
&	AND
isTRUE (x)	test if x is TRUE

Creating New Variables

There are three different assignment operators in R. The operators "<-" and "=" assign into the environment in which they are evaluated. The operator "<-" can be used anywhere, whereas the operator "=" is only allowed at the top level (e.g., in the complete expression typed at the command prompt) or as one of the sub-expressions in a braced list of expressions.

The operator "<<-" is normally only used in functions, to access variables in the global environment instead of the local environment.

"<-" and "<<-" have both leftward and rightward forms.

Packages

- Applications of **R** normally use a **package**; i.e., a library of special functions designed for a specific problem.
- Most predefined functions are in the base package.
- A user normally only loads a handful of packages for a particular analysis.
- To use a package, should be downloaded and installed once. Then `library(package)` loads the package to the current session.

Fundamental Data Objects

- **vector** - a sequence of numbers or characters, or higher-dimensional arrays like matrices.
- **list** - a collection of objects that may themselves be complicated.
- **factor** - a sequence assigning a category to each index.
- **data.frame** - a table-like structure (experimental results often collected in this form).
- **environment** - hash table. A collection of key-value pairs.

Classes of objects are built from these. Most commands in **R** involve applying a **function** to an object.

Unlike **C** variables do not need to be declared and typed. For example, assigning a sequence of numbers to `x` forces `x` to be a numeric vector.

Given `x`, executing `class(x)` reports the class. This indicates which functions can be used on `x`.

Vectors

In R the “base” type is a vector, not a scalar. A vector is known as an array in programming languages. A vector is an indexed set of values that are all of the same type. The type of the entries determines the class of the vector. The possible vectors are:

- numeric
- integer (subclass of numeric)
- character
- complex
- logical

Vectors of different classes cannot be combined. Entries in a vector can be “named”.

- Numeric vectors can be used in arithmetic expressions, in which case the operations are performed element by element to produce another vector.
- Missing values in a vector will be denoted by NA.
- Can subset with a logical vector.

Factor

Typically, in an experiment samples are classified into one of a set group of categories. In **R** such results are stored in a **factor**. A factor is a character vector augmented with information about the possible categories, called the **levels** of the factor.

Lists

A list is an ordered collection of objects. Unlike vectors, lists can be used if we want to create collections of vectors or other data objects of mixed type. The objects in a list are known as its **components**.

Matrix

Can be used to represent matrices in R. It is same as in vectors, all elements should be of the same type.

Data Frame

A data.frame object in R has similar dimensional properties to a matrix but it may contain different types of data in it. A list can be made into a `data.frame` when,

- Components are vectors (numeric, character, logical) or factors.
- All vectors and factors must have the same lengths.
- Elements in a data frame are indexed like a matrix.
- Columns can be accessed using the column names as in lists (using \$ operator).
- Character vectors in a data frame are always stored as a factor.

Exercise

Instructions: Create a folder in your desktop with your registration number (Eg: "IT....."). You need to save the R script file and take screenshots of the command prompt with answers and save it in a word document inside the folder. Save both R script file and word document with your registration number (Eg: "IT....."). After you finish the exercise, zip the folder and upload the zip file to the submission link.

1. Open R or RStudio and type the following at the command prompt to perform operations:

```
a <- 10  
b <- 3
```

2. Use R to compute and print the results for the followings:

```
a + b  
a %/% b  
a %% b  
a ^ b
```

3. Use logical operators to test the following and print results:
 - a) Is *a* greater than *b*?
 - b) Is *b* equal to 3?
 - c) Is either *a* > 5 or *b* < 2 true?

4. Create the numeric vector below:

```
scores <- c(85, 90, 78, 92, NA, 88)
```

Find,

- the class of the vector.
 - the mean of the scores (use na.rm = TRUE).
 - Which scores are above 85?
5. Create a character vector of student names and a logical vector indicating pass/fail as below:

```
names <- c("Alice", "Bob", "Charlie", "Diana", "Evan", "Fiona")
```

```
passed <- scores >= 80
```

6. Create a data frame called “results” and combine the vectors into the data frame as below:

```
results <- data.frame(Name = names, Score = scores, Passed = passed)
```

7. View the structure and contents of the data frame using following commands:

```
str(results)
```

```
print(results)
```

8. Access the Score column using the \$ operator and find the maximum score.